

Using RStudio

Learning objectives

1. To be able to identify and use the following tabs in RStudio:

- Source,
- Console,
- Environment,
- History,
- Files,
- Plots,
- Packages,
- Help,
- Viewer.

2. To be able to modify RStudio's interface options to suit your needs.

RStudio Environment

- Now that you have access to R & RStudio, let's go on a quick tour of the RStudio interface, and learn how to use the software.
- In this section, the goal is to help you get acquainted with the tools you can use in RStudio.

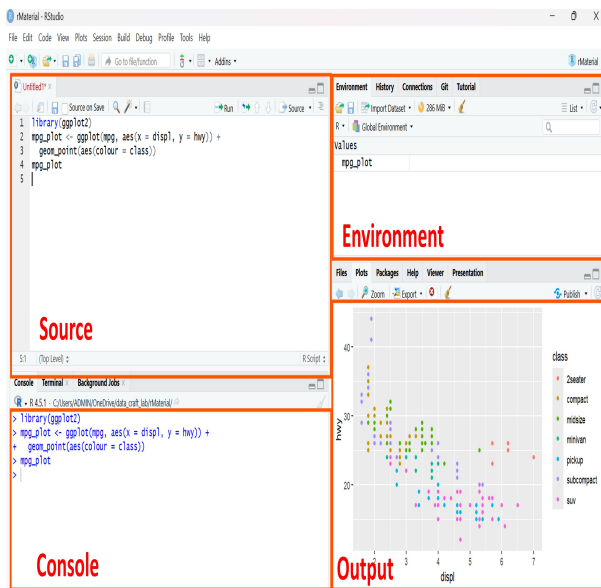
- To get started, you need to open the RStudio application:

Take note!

R is case-sensitive: That is, the name **DENGUE** in R will not be the same as **dengue** nor **Dengue** nor **denguE**. So, while using the software commands or names of objects, this should be checked to avoid having errors

The RStudio panes

- By default, RStudio is arranged into four window panes as shown below.

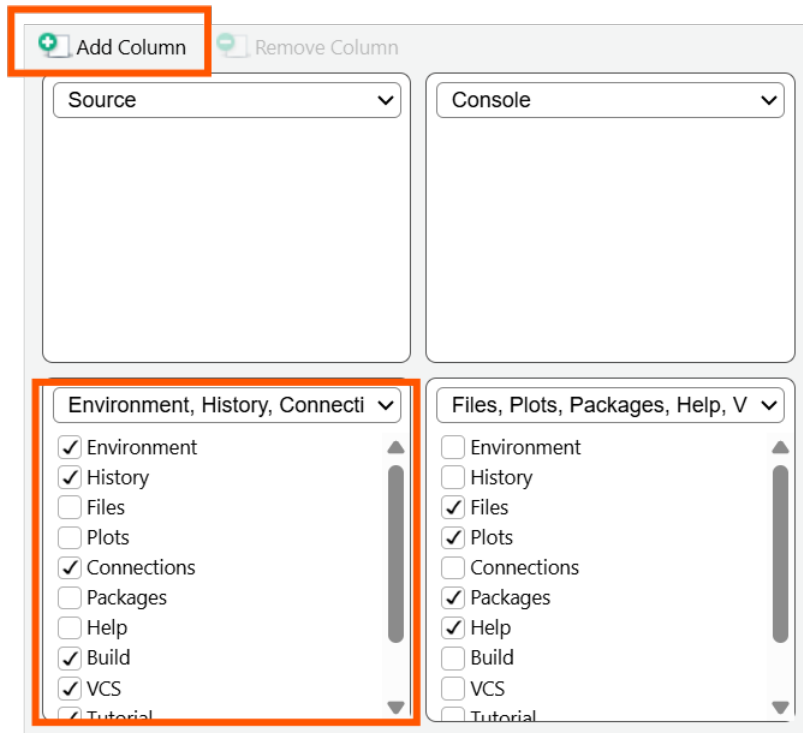


- If you only see three panes, then this means that the source pane is missing, thus, you can click to open a new script with **File > New File > R Script**. This should reveal one more pane.



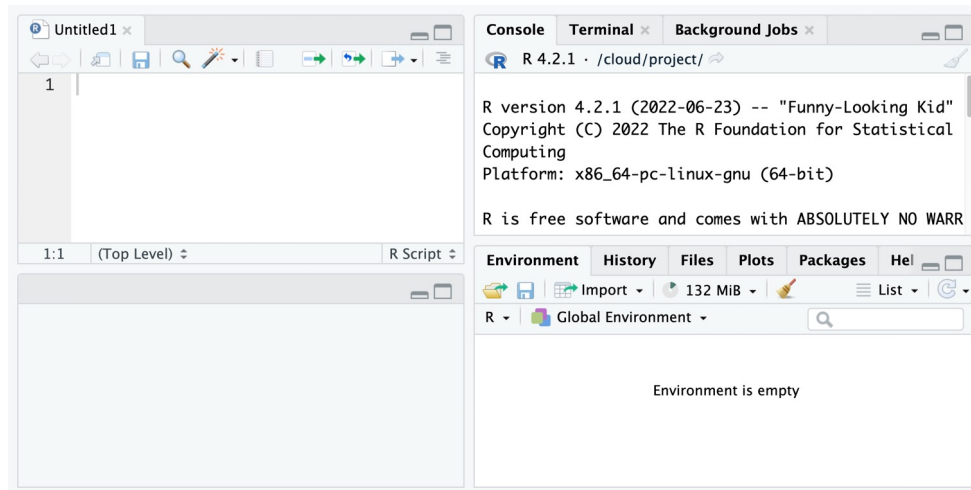
- To improve on the usability of the interface, we can rearrange these panes as much as we can.

- To do this, in the RStudio menu at the top of the screen, select Tools > Global Options to bring up RStudio's options. Then under Pane Layout, adjust the pane arrangement. The arrangement should be as shown below.

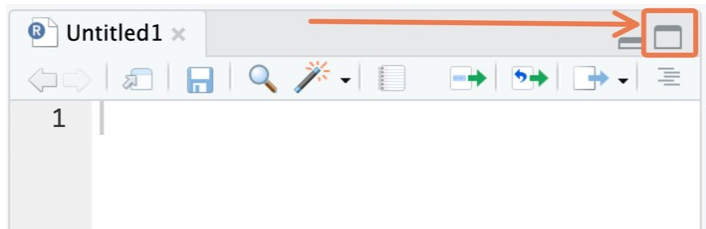


- At the **top left** pane is the Source tab, and at the top right pane, you should have the Console tab.
- Then at the **bottom left** pane, no tab options should be checked—this section should be left empty, with the drop-down saying just “TabSet”.
- Finally, at the **bottom right** pane, you should check the following tabs: Environment, History, Files, Plots, Packages, Help and Viewer.

- Great, now you should have an RStudio window that looks something like this (although this depends on your choice of windows):



- The top-left pane is where you will do most of the coding. Make this larger by clicking on its maximize icon:



- Note that you can drag the bar that separates the window panes to resize them.



- Now let's look at each of the RStudio tabs one by one. Below is a summary image of what we will discuss:

Editor

Type and save code as scripts

```
rstudio_intro.R x women x
1 print("excited for R")
2 print("and RStudio")
3
4 View(women)
5
6 ebola_data <- read.csv("https://tinyurl.com/ebola")
7
8 plot(women)
9
```

Console

Run code

```
R 4.2.1 · /cloud/project/
[1] "and RStudio"
> View(women)
> 2 + 2
[1] 4
> 3 + 3
[1] 6
> ebola_data <- read.csv("https://tinyurl.com/ebola")
> plot(women)
>
```

Environment

History

Files

Plots

Packages

Help

Environment

View datasets & other objects in workspace

199 MiB	
Global Environment	
Data	
ebola_data	200 obs. of 7 variabl...
\$ id	: int 167 129 2...
\$ age	: num 55 41 12 ...

History

View and search through previous commands

```
print("excited for R")
print("and RStudio")
View(women)
2 + 2
3 + 3
print("excited for R")
```

Files

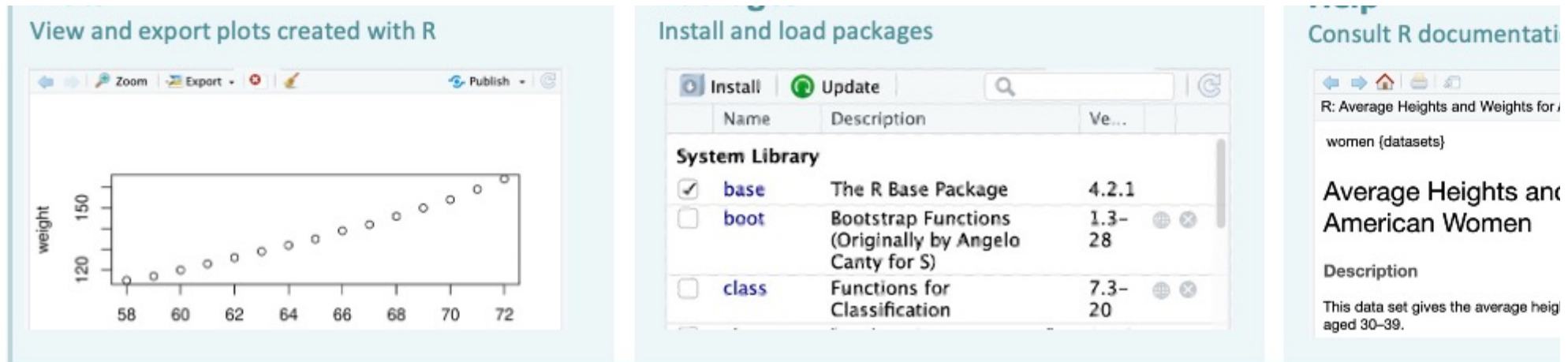
Interact with the comp

Cloud	project
Name	
..	
.Rhistory	
project.Rproj	
rstudio_intro.R	

Plots

Packages

Help



Source/Editor

```
1 print("excited for R!")
2 print("and RStudio")
```

- The source or editor is where your R “scripts” go. A script is a text document where you write and save code.
- Because this is where you will do most of your coding, it is important that you have a lot of visual space. That is why we rearranged the RStudio pane layout above—to give the Editor more space.
- Now let’s see how to use this Editor.

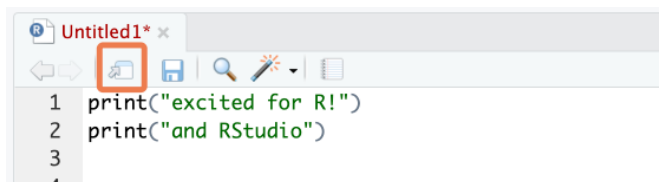
- First, **open a new script** under the File menu if one is not yet open: File > New File > R Script. In the script, type the following:
- You can also **run multiple lines at once**. To try this, add a second line to your script, so that it now reads:

```
print("excited for R!")  
print("and RStudio!")
```

- Now drag your cursor to highlight both lines and press Command/Control + Enter.
- To **run the entire script**, you can use Command/Control + A to select all code, then press Command/Control + Enter. Try this now. Deselect your code, then try to the shortcut to select all.

An easier way of doing this is to use the 'Run' button at the top right of the source panel, with which you can run code (either the current line, or all highlighted code)  Run.

- To **open the script in a new window**, click on the third icon in the toolbar directly above the script.

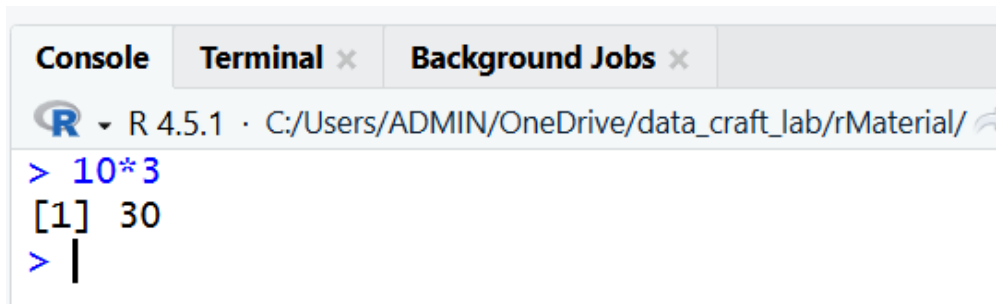


- To put the window back, click on the same button on the now-external window.

- Next, **save the script.** use file-save as or just hit Command/Control + S to bring up the Save dialog box. Give it a file name of your choice but a meaningful name.
- If you are working with RStudio cloud, the file will be saved in your **working directory.**
- If you are working on your local computer, save the file in an easy-to-locate part of your computer, perhaps your desktop.

Console

- The *console*, at the bottom left (or top right), is where **code is executed**. You can type code directly here, **but it will not be saved**.
- Type a random piece of code (maybe a calculation like `10 + 4`) and press 'Enter'.

A screenshot of the RStudio interface's console pane. At the top, there are three tabs: 'Console' (selected), 'Terminal', and 'Background Jobs'. Below the tabs, the console shows the R prompt '>' followed by the code '10*3'. The output is '[1] 30'. Below the output, the prompt '>' is followed by a vertical bar cursor '|'. The console title bar indicates 'R 4.5.1' and the current working directory 'C:/Users/ADMIN/OneDrive/data_craft_lab/rMaterial/'.

If you place your cursor on the last line of the console, and you press the **up arrow**, you can go back to the last code that was run. Keep pressing it to cycle to the previous lines.

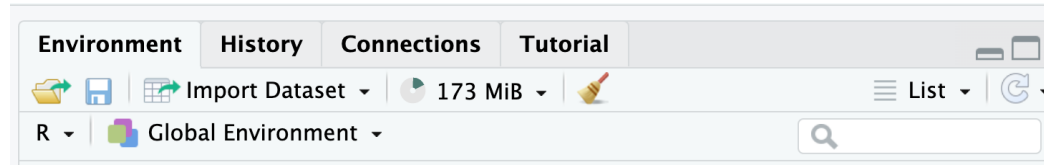
To run any of these previous lines, press *Enter*.

- Other tabs seen on this pane include **Terminal**, **Render**, **Background Jobs**. These are also used by R to accomplish other tasks that we will see later as we learn R.

Environment

- Initially, when you open RStudio, you should see at the top right of the RStudio Window, the

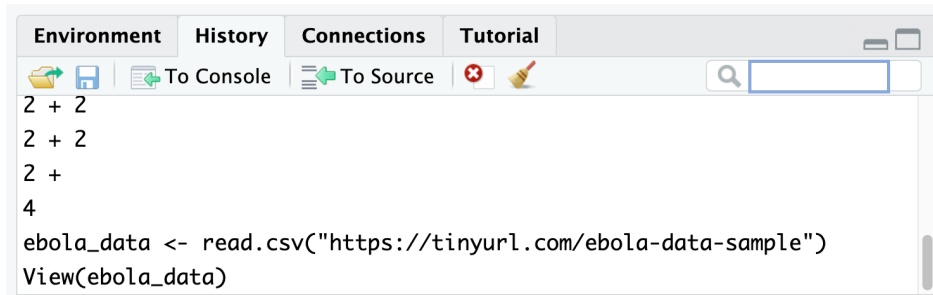
Environment tab.



- The Environment tab shows datasets and other objects that are loaded into R's working memory, or “workspace”.
- We will see how to explore this tab later when we will be improving datasets.

History

- Next, the **History** tab shows previous commands you have run.

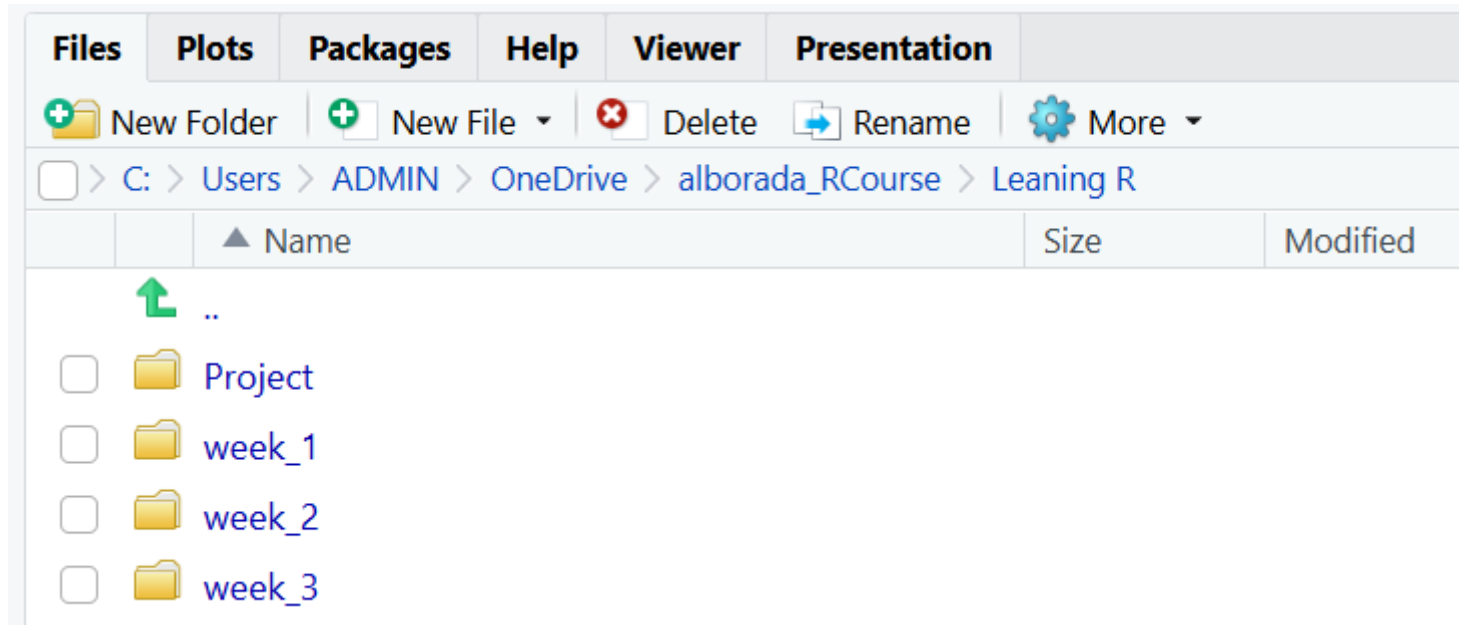


- You can click a line to highlight it, then send it to the console or to your script with the “To Console” and “To Source” icons at the top of this tab.
- To select multiple lines, use the “Shift-click” method: click the first item you want to select, then hold down the “Shift” key and click the last item you want to select.

- Finally, notice that there is a search bar at the top right of the History pane where you can search for past commands that you have run.

Files

- Next, the **Files** tab. This shows the **files and folders in the folder** you are working in.



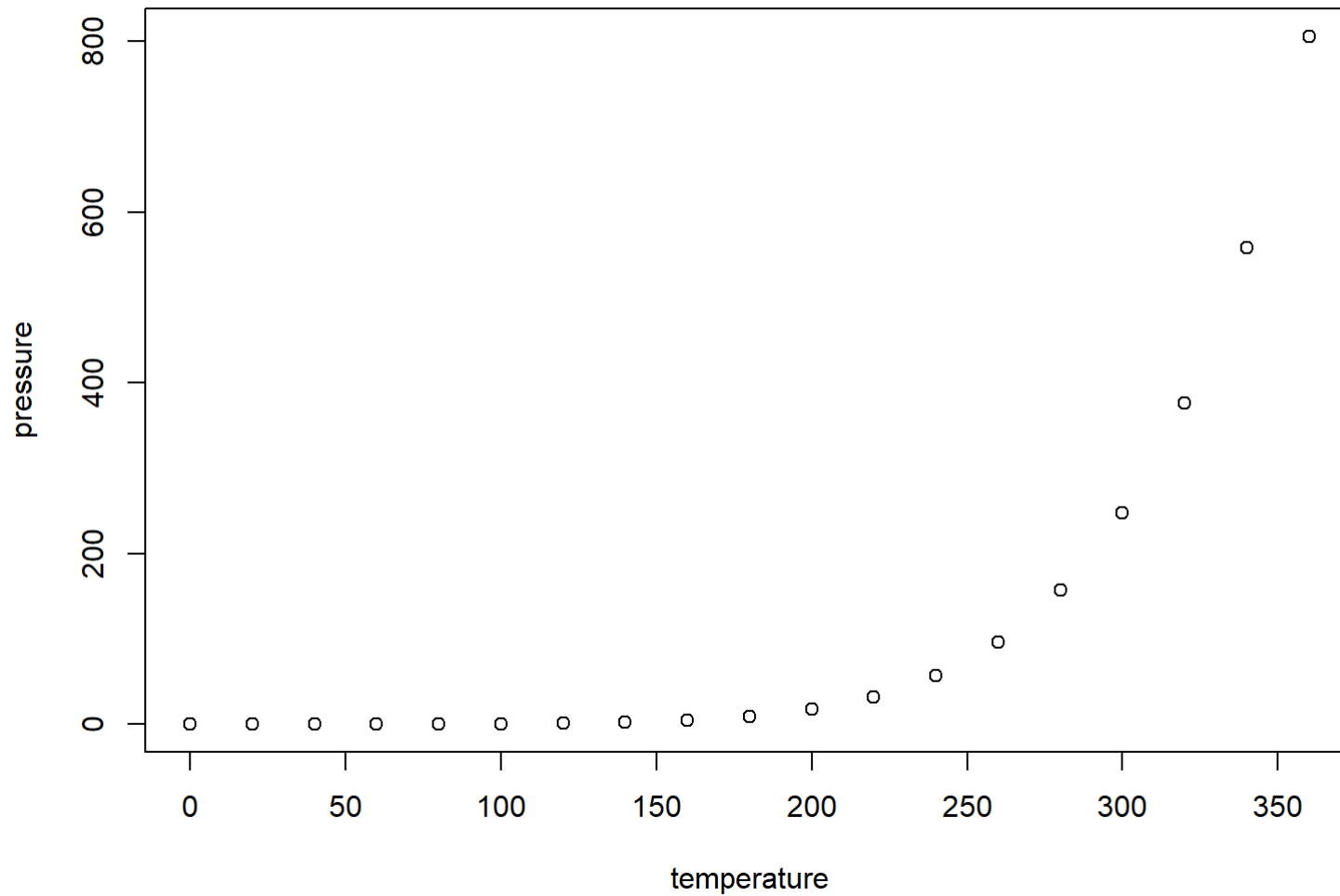
- The file tab allows you to interact with your computer's file system easily.
- Try playing with some of the buttons here, to see what they do. You should try at least the following:
 - *Make a new folder*

- *Delete that folder*
- *Make a new R Script*
- *Rename that script*

Plots

- Next, the **Plots** tab. This is where figures that are generated by R will show up. Try creating a simple plot with the following code:

```
plot(pressure)
```

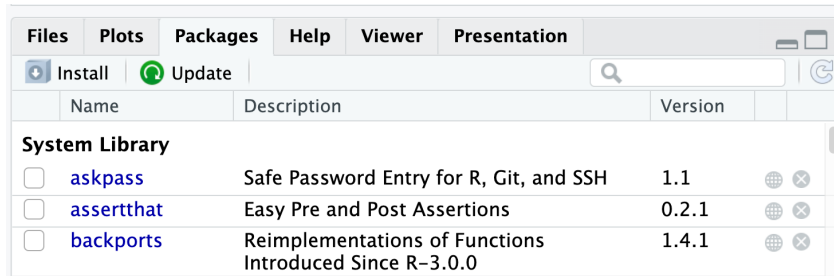


- That code creates a plot of the two variables in the pressure dataset, which is a base R in build dataset. You should see this figure in the Plots tab.

- Now, test out the buttons at the top of this tab to explore what they do (e.g zoom). In particular, try to export a plot to your computer.

Packages

- Next, let's look at the **Packages** tab.



- **Packages are collections of R code that extend the functionality of R.** More details will be provided on packages as we progress.

Base R packages

- Knowing what **base R packages** are available helps you understand which functions and datasets come with R by default (without needing to install anything)
- To see the base R, packages, we can run the following code

```
installed.packages(priority = "base")
```

- This shows core R packages that are essential for R to run (like stats, utils, graphics, etc.).

- To include the “recommended” ones (those that come preinstalled but can be updated), use:

```
installed.packages(priority = c("base", "recommended"))
```

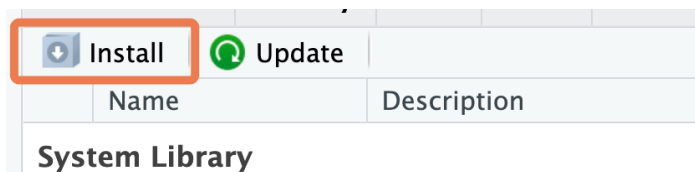
Session packages

- To get info on the packages available in the current session, use the code

```
sessionInfo()
```

Package installation

- For now, it is important to know that to use a package, you need to *install* then *load* it. **Packages need to be installed only once, but must be loaded in each new R session.**
- All the package names you see (in blue font) are packages that are installed on your system. And packages with a checkmark are packages which are *loaded* in the current session.
- You can install a package with the Install button of the Packages tab.



- But it is better to install and load packages with R code, rather than the Install button. Let's try this. Type and run the code below to install the {pacman} package.

```
install.packages("pacman")  
library(pacman)
```

- The first line installs the package. The second line *loads* the package from your package library.
- Because you only need to install a package once, you can now remove the installation line from your script (see *commenting in R*).
- Packages that have already been installed can also be updated. You can either update a single package or all packages already installed in R
- Most packages in R come with datasets, and thus one can use the datasets for analysis

Viewer

- Notice that the histogram above shows up in a **Viewer** tab. This tab allows you to preview HTML files and interactive objects.

Help

- Lastly, the **Help** tab shows the documentation for different R objects. Try typing out and running each line below to see what this documentation looks like.

[?data](#)
[?read.csv](#)

R: Data Input ▾ Find in Topic

read.table {utils} R Documentation

Data Input

Description

Reads a file in table format and creates a data frame from it, with cases corresponding to lines and variables to fields in the file.

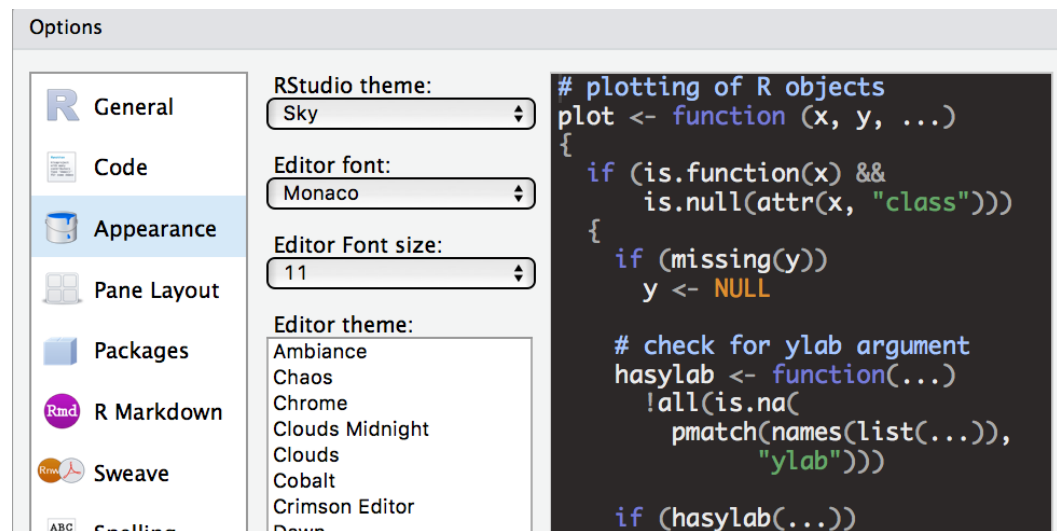
Usage

```
read.table(file, header = FALSE, sep = "", quote = "\"\"",  
  dec = ".", numerals = c("allow.loss", "warn.loss", "no.l  
  row.names, col.names, as.is = !stringsAsFactors, tryLogi  
  na.strings = "NA", colClasses = NA, nrows = -1,
```

- Help files are not always very easy to understand for beginners, but with time they will become more useful.

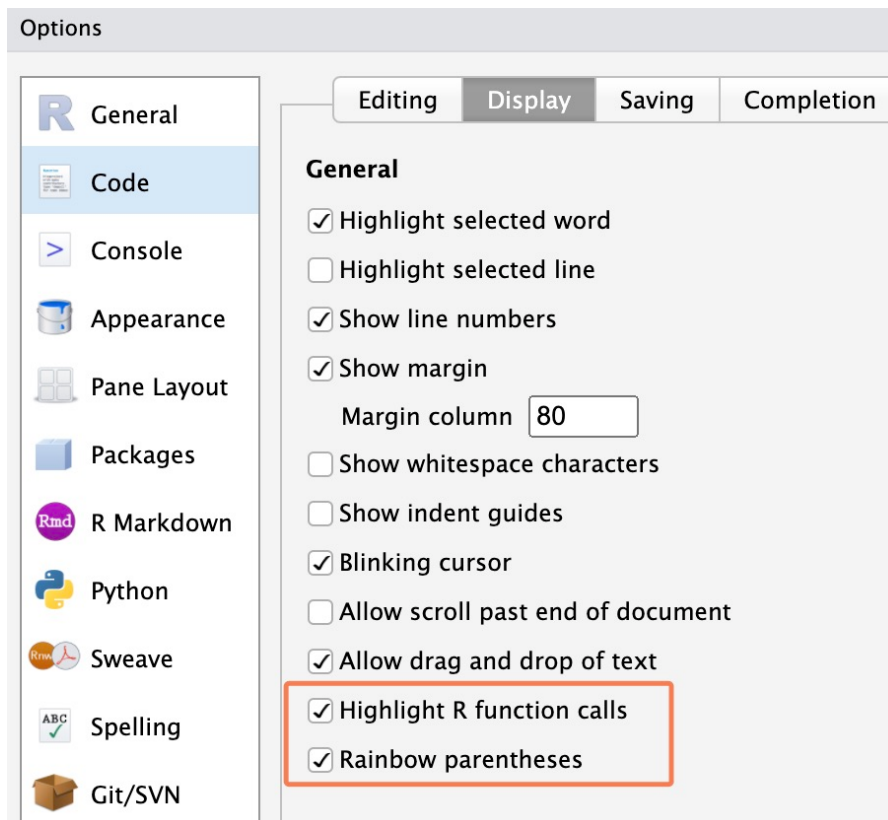
RStudio options

- RStudio has a number of useful options for changing its look and functionality. Let's try these.
- In the RStudio menu at the top of the screen, select Tools > Global Options to bring up RStudio's options.
- Now, under Appearance, choose your ideal theme. (I have chosen the “Crimson Editor” and “Tomorrow Night” themes.)



- Under Code > Display, check “Highlight R function calls”. What this does is give your R *functions* a unique color, improving readability. This will come clear later as we progress.
- Also under Code > Display, check “Rainbow parentheses”. What this does is make your “nested

parentheses” easier to read by giving each pair a unique color.



`print("excited for R")`
`View(cars)`

Highlight R function calls

`print("excited for R")`
`View(cars)`

`((1 + 1) * 2)^2`

Rainbow parentheses

`((1 + 1) * 2)^2`

- Under the Code > Display, you can also adjust the margin of the code from the default 80 to

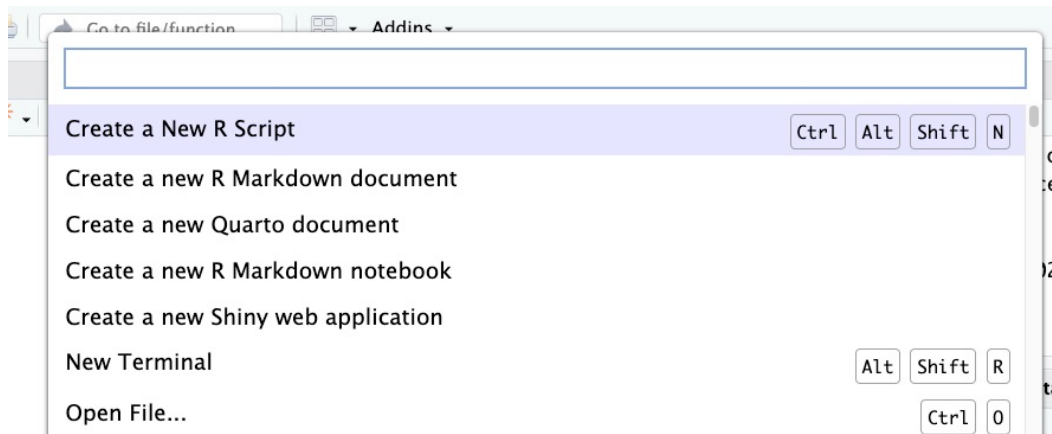
another number of choice.

- Finally under General > Basic, **uncheck** the box that says “**Restore .RData into workspace at startup**”. You don’t want to restore any data to your workspace (or *environment*) when you start RStudio. Starting with a clean workspace each time is less likely to lead to errors.

This also means that you never want to “**save your workspace to .RData on exit**”, so set this to **Never**.

Command palette

- The Rstudio command palette gives instant, searchable access to many of the RStudio menu options and settings that we have seen so far.
- The palette can be invoked with the keyboard shortcut `Ctrl + Shift + P` (`Cmd + Shift + P` on macOS).
- It's also available on the *Tools* menu (*Tools -> Show Command Palette*).



Try using it to:

- Create a new script (Search “new script” and click on the relevant option)
- Rename a script (Search “rename” and click on the relevant option)

Wrapping up

- As you advance in your R journey, you will discover new features, and you will hopefully grow to love the wonderful integrated development environment (IDE) that is RStudio.
- One good place to start is the official RStudio IDE [cheatsheet](#).

Below is one section of that sheet:

RStudio IDE Cheat Sheet

learn more at www.rstudio.com



The RStudio IDE is an Integrated Development Environment in R that comes in three versions



Desktop IDE

A local version of the IDE for your desktop



Open Source Server

for larger compute resources and remote access



Professional for team resources and environment

Download all at www.rstudio.com. Each provides the same useful interface:

Documents and Apps

Open Shiny, R Markdown, knitr, Sweave, LaTeX, .Rd files and more in Source Pane

Check spelling, Render output, Choose output format, Choose output location, Insert code chunk

Jump to previous chunk, Jump to next chunk, Run selected lines, Publish to server, Show file outline

Access markdown guide at **Help > Markdown Quick Reference**

Jump to chunk, Set knitr chunk options, Run this and all previous code chunks, Run this code chunk

RStudio recognizes that files named **app.R**, **server.R**, **ui.R**, and **global.R** belong to a shiny app

Run app, Choose location to view app, Publish to shinyapps.io or server, Manage publish accounts

Write Code

Navigate tabs, Open in new window, Save, Find and replace, Compile as notebook, Run selected code

Cursors of shared users, Re-run previous code, Source with or without Echo, Show file outline

Multiple cursors/column selection with **Alt + mouse drag**

Code diagnostics that appear in the margin. Hover over diagnostic symbols for details.

Syntax highlighting based on your file's extension

Tab completion to finish function names, file paths, arguments, and more.

Multi-language code snippets to quickly use common blocks of code.

Jump to function in file, Change file type

Working Directory, Maximize, minimize panes, Press **↑** to see command history, Drag pane boundaries

R Support

Import data file with wizard, History of past commands to run/add to source, Display .Rpres slideshows **File > New File > R Presentation**

Load workspace, Save workspace, Delete all saved objects, Search inside environment

Choose environment to display from list of parent environments, Display objects as list or grid

Displays saved objects by type with short description, View in data viewer, View function source code

Create folder, Upload file, Delete file, Rename file, Set As Working Directory, Go To Working Directory, Change directory

Path to displayed directory

A File browser keyed to your working directory. Click on file or directory name to open.

RStudio IDE

Share Project with Collaborators, Active collaborators

Name of current project

RStudio opens plots in GUI

Navigate recent plots, Open in new window

GUI Package manager

Install Packages, Update Packages, Create Library

Click to load package with **library()**. Unclick to detach package with **detach()**

Debug Mode

Open with **debug()**, **browse()**, or a breakpoint. RStudio will open the debugger mode when it encounters a breakpoint while executing code.

Launch debugger mode from origin of error, Open traceback to examine the functions that R called before the error occurred

Click next to line number to add/remove a breakpoint

Error in `get_digit(num, x)`: Show Traceback

Version Control with Git or SVN

Turn on at **Tools > Project Options > Git/SVN**

Stage files, Show file diff, Commit staged files, Push/Pull to remote, View History

Added, Deleted, Modified, Renamed

Open shell to, current

RStudio opens documents

Home page of helpful links, Search help file

Viewer Pane displays HTML R Markdown reports, an

Highlighted line shows where execution has paused

Run commands in environment where execution has paused

Examine variables in executing environment

Select function in traceback to debug

Step through code one line at a time

Step into and out of functions to run

Resume execution mode

Quit debug

Package Writing

File > New Project > New Directory > R Package

Turn project into package, Enable roxygen documentation with **Tools > Project Options > Build Tools**

Roxygen guide at **Help > Roxygen Quick Reference**

Stop Shiny app

View(<data>) opens spreadsheet

	Sepal.Length	Sepal.Width
1	5.1	3.1
2		
3		
4		

Filter rows by value or value range